Benchmarks as Software: Continuous Validation and a Patch for Terminal-Bench 2.0

E. Kelly Buchanan [1] **Scott W. Linderman** [1] **Christopher Ré** [1]

Abstract

Coding agents are among the most economically consequential deployments of LLMs to date. Reliably evaluating agentic capabilities is a complex problem; even benchmarks that are thoroughly validated at release can become stale as issues accumulate into *benchmark maintenance debt*, a growing backlog of defects that decouple a model's score from the capability it is meant to measure. To address this, we propose a Continuous-Validation Workflow for agentic benchmarks that combines community reports, task files, agent traces, automated oracle checks, and human-gated repairs to audit and repair benchmarks. We apply this workflow to Terminal-Bench 2.0, whose creation involved significant human-and-LM review and red-teaming. Surprisingly, the audit surfaces defects in 28/89 tasks, showing that *a third of the benchmark needs maintenance due to different sources of drift*. We ship the fixes as Terminal-Bench 2.1. In Terminal-Bench 2.1, individual agent accuracy changes by up to 12 pp, and leaderboard rankings move up to 3 positions (Spearman $\rho = 0.958$). Beyond accuracy, we evaluate whether benchmark repair changes efficiency conclusions. For this, we compare model performance across two axis: agent harness fit and compute budgets. Overall our results show that benchmark debt is pervasive and that these issues affect not only who wins, but how expensive winning appears to be.

1. Introduction

Agentic benchmark scores shape product claims and research investment. But agentic benchmarks are not static datasets: over time, these systems accumulate defects that decouple a model's score from the capability the benchmark is meant to measure. In turn *benchmark maintenance debt*: the backlog of task-side defects that must be repaired or retired for benchmarks to remain valid is accumulated along the following axes:

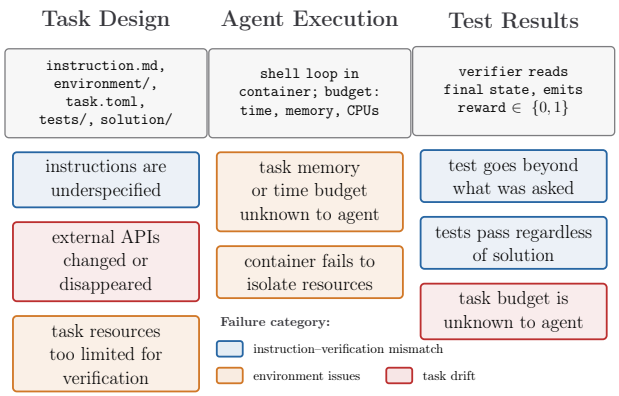


Figure 1. **Failure surfaces along the agentic-task lifecycle**. From authoring to execution and verification, each stage can admit a failure. Colors mark the three failure categories (blue = instruction–verification mismatch, orange = environment issues, red = task drift).

- **Fidelity debt**: tasks and verifiers encode assumptions about how agents will behave, which break when agents discover shortcuts such as hardcoded solutions, substring-matching exploits, or valid solution paths the author did not anticipate.
- **Drift debt**: live dependencies, websites, APIs, and package ecosystems change after release, causing tasks to fail for reasons unrelated to model capability. For agentic benchmarks, validity is therefore not a one-time certificate, it is a maintenance problem.

The problem can be illustrated with Terminal-Bench 2.0 (Merrill et al., 2026), an agentic benchmark reported in model cards from OpenAI (OpenAI, 2024a), Anthropic (Anthropic, 2026a), Cursor (Cursor Research Team, 2026), Qwen (Qwen Team, 2026), GLM (Zeng et al., 2026), and Google (Google DeepMind, 2026), among others. Terminal-Bench 2.0 tasks were manually reviewed, LM-audited, and exploit-tested before release to ensure quality. However release-time validation is not enough: within months, the benchmark had accumulated over 50

[1]Stanford University, Stanford, CA, USA. Correspondence to: E. Kelly Buchanan <kelly.buchanan@stanford.edu>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

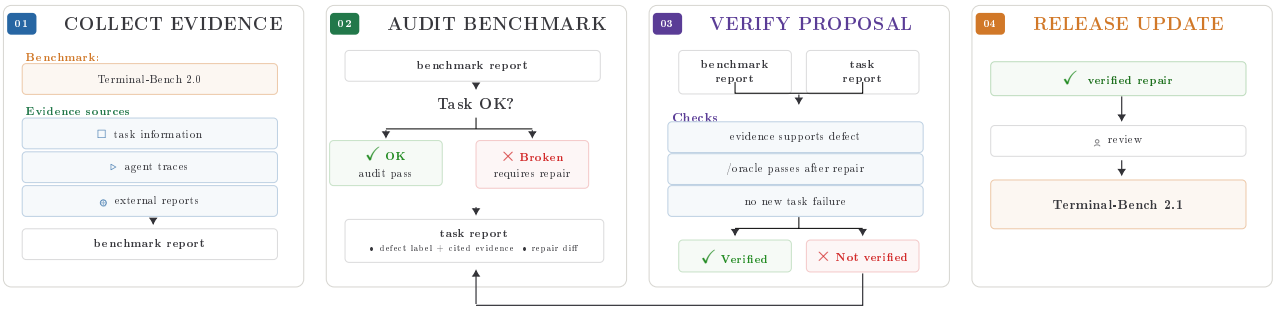


Figure 2. **Agentic pipeline for continuous benchmark validation.** An evidence collector agent gathers complementary evidence streams: static task files, prior agent traces, and external reports. An auditor agent applies audit principles for specification fidelity, tests, resources, and risk, then drafts a cited repair, which is checked against evidence while `/oracle` rebuilds the task and runs the reference solution. The repair workflow can open a draft PR with evidence; maintainer review gates the verified release.

open GitHub issues tagged as task defects Github (2026a;b); Zeng et al. (2026), which spanned implicit assumptions made across the full task stack (Fig. 1).

The resulting repair proposals varied in quality. Some addressed the underlying validity failure by clarifying the task or repairing the oracle; others only patched the observed symptom, for example by relaxing a regex or broadening an acceptance condition. Infrastructure variance in VM providers, hardware, and timeout behavior further entangled task defects with the systems used to run agents (Segato, 2026; OpenAI, 2025b),

Existing responses address parts of this problem. One response is to freeze the environment. Freezing improves reproducibility, but creates fidelity debt: the benchmark becomes stable by becoming less like the live setting it was meant to model. A second response is to filter rather than repair, evaluating only on a validated subset (OpenAI, 2024b). Filtering improves precision, but agentic tasks are expensive software artifacts and removing them reduces coverage and can change the benchmark's difficulty distribution. A third response is to refresh the benchmark continuously (Jain et al., 2025). Refreshing addresses contamination and staleness, but it does not answer the post-release maintenance question: when defects are discovered, which tasks can be repaired rather than filtered, what validation is sufficient for each repair, and how much do these decisions change reported accuracy, rankings, and cost?

We address benchmark maintenance debt in three steps:

- **A continuous validation workflow.** We show that benchmark validation can itself be continuous: modern agentic tools make it practical to combine community reports, public execution traces, agent-assisted triage, and human adjudication into a maintainable workflow for identifying, repairing, retiring, and documenting benchmark issues (Sec. 4)
- **An updated benchmark, Terminal Bench 2.1.** We apply this workflow to Terminal-Bench 2.0 to release Terminal-Bench 2.1. Across the 89 tasks in Terminal-Bench 2.0, the workflow surfaces issues in roughly one third of the benchmark. We categorize these issues into three recurring sources of maintenance debt: *environment issues*, *instruction–verification mismatches*, and *task drift* (Sec. 5).
- **The effects of benchmark repair.** We re-evaluate 19 agents on Terminal-Bench 2.0 and Terminal-Bench 2.1 and show that repair can change reported accuracy and efficiency conclusions without fully overturning the leaderboard. Agent rankings remain highly correlated, but individual scores move by nontrivial margins. (Sec. 6)

2. Related Work

Benchmark auditing. Prior work has studied how benchmark conclusions change when the evaluation set is revised. Recht et al. (2019) resampled ImageNet and CIFAR test sets and used rank correlation between the original and new test sets to evaluate whether a benchmark measured is a stable construct. We adopt the same rank-stability protocol, but apply it to a different failure mode: task-side defects in an executable agent benchmark.

A second response is to validate and filter benchmark instances. SWE-Bench Verified (OpenAI, 2024b) contracted professional developers to audit SWE-Bench (Jimenez et al., 2024) and released a conservatively filtered subset. WebArena Verified (ServiceNow Research, 2026) follows a similar validated-subset approach for web-agent tasks. Related audits identify solution leakage, weak test suites, and task/outcome-validity failures in agentic benchmarks (Aleithan et al., 2024; Yu et al., 2025; Zhu et al., 2025). These approaches typically treat validation as a one-shot curation pass and reduce the benchmark rather than repairing it.

A third response is to time-window or refresh benchmarks. LiveCodeBench (Jain et al., 2025) and LiveBench (White et al., 2025) continuously add new problems to reduce

contamination and benchmark staleness. This approach is closest in spirit to continuous validation, but instead we focus on repairing rather than filtering.

Auditing tooling. Benchmark auditing is facilitated by a growing stack of operational tools. At the rubric layer, the Agentic Benchmark Checklist (ABC) (Zhu et al., 2025) provides an actionable framework for auditing outcome validity and task validity. At the evidence-collection layer, tools such as Docent (Meng et al., 2025) and Langsmith (LangChain, 2026) make large-scale inspection tractable by ingesting benchmark trajectories for reviewing. Complementary adversarial tools audit benchmark hackability by searching for ways that agents can pass tests without satisfying the intended task.

These tools address different parts of the audit pipeline. Rubrics specify the evaluation criteria; ingestion tools collect evidence; red-teamers identify candidate defects; and human adjudication resolves ambiguous cases. We combine these ingredients into a continuous validation workflow (Sec. 4).

Agentic Evaluation Executable agent benchmarks do not evaluate a model in isolation: scores also depend on the agent harness, execution environment, and resource budget used to run the model. For example, AgentBench (Liu et al., 2024) documented that scaffold choice can move benchmark scores by tens of percentage points at fixed model. SWE-Agent (Yang et al., 2024), Mini-SWE-Agent (SWE-agent Team, 2025), OpenHands (Wang et al., 2025), Terminus 2 (Merrill et al., 2025), Claude Code (Anthropic, 2025), Codex CLI (OpenAI, 2025a), and Gemini CLI (Google, 2025) all report gains. Terminal-Bench 2.0 showed that scaffold choice is less impactful than model choice on average, but this can obscure within-family harness effects (Merrill et al., 2026).

3. Problem setup

Every task has a failure surface. A Terminal-Bench task is a timed exam with the the following components:

- **Instruction:** the natural-language problem description shown to the agent.
- **Environment:** the Docker image with pre-installed software, network rules, and any provided files.
- **Oracle:** the known-correct solution.
- **Test:** the grading code that runs after the agent stops.
- **Resources:** CPU, memory, and wall-clock budgets.

For each task, the five components are specified by the task contributors (Merrill et al., 2026).

An *agent* is a model with access to tools. The *agent harness* is the system that gives a model the tools interact with its environment. A *trial* is one independent attempt at a task by an agent.

For agent a working on task t with $n_{a,t}$ trials, $\hat{p}_{a,t}$ is the mean reward. The coverage, or *pass@k*, estimates the probability that at least one of the trials succeeds. The coverage is the unbiased pass@k estimator from Chen et al. 2021 throughout.

Datasets. We operate with two datasets: the *evidence data* and the *evaluation data*. The evidence data is the external evidence gathered on the benchmark, including trial data from the leaderboard, used to identify and fix defective tasks and release Terminal Bench 2.1. The evaluation data is a fresh set of TB2.0 and TB2.1 trials, run for 19 agents, used to measure the effect of the fixes.

To isolate the role of efficiency, we run an additional condition on both TB2.0 and TB2.1 that deliberately deviates from the benchmark guidelines: we increase the resources allocated, giving agents more time to complete each task. Owing to cost, this condition is run on a subset of the 19 agents.

4. Benchmark Auditing Workflow

Our goal is to audit agentic benchmarks for task-side defects and to produce candidate repairs that can be reviewed by maintainers. We have at our disposal three evidence streams: the task's specification, prior execution traces, and community reports from the web. Given these evidence streams, we want this information to be compiled into a task-level audit report: a defect label, supporting evidence, and a proposed fix when one is available. We define three agent skills for collecting evidence, auditing, and verifying the final report (OpenAI, 2024b).

Audit rubric. A central requirement of the workflow is a fixed rubric. Without one, an agent can flag suspicious behavior, but its behavior may not be consistent or easy for a human to parse. Previous work has rubrics developed to audit benchmarks (Zhu et al., 2025). We build on this idea, but adapt it for a maintenance workflow. In this setting, we found it more useful to ask for a small number of action-oriented defect labels tied to likely repairs, rather than broad free-form validity judgments. The auditor therefore assigns a primary task-side defect category and records secondary labels when applicable. This makes reports comparable across agents and gives maintainers a concrete review target. We ask auditors to classify each task using the following primary *defect categories*:

- **No defect found:** the task is well specified, the oracle is accepted, the verifier checks the intended behavior, and the stated resource budget is sufficient.
- **Instruction–verification mismatch:** the task instruction and the verifier disagree. This includes underspecified instructions whose hidden constraints appear only in tests, as well as overspecified tests that reject behavior satisfying the task as written.
- **Task drift:** the task changed after release because of

mutable external state, such as package versions, network resources, public APIs, or web content.
- **Environment issues:** the task is solvable in principle, but the provided environment and resources (wall-clock, CPU, memory, or filesystem/network configuration) prevents the oracle or verifier from running reliably.

Audit workflow The `audit` workflow is illustrated in Fig. 2. For each task, the *evidence* agent produces a structured audit report containing information on the benchmark, including task specification, trace information, and community reports. The role of this agent is to organize available evidence for a downstream auditor, while keeping observation separate from adjudication.

An *auditor* applies the rubric to each task. The auditor checks whether the task is well specified, solvable, and verifiable; whether the verifier accepts valid solutions and rejects invalid ones. The rubric here follows the ABC guidelines (Zhu et al., 2025), and it produces a task-level report containing a primary defect label, a short justification, the location of the defect, and a proposed repair.

A *verifier* reviews the evidence and the auditor's label. We include this stage because evidence collection and auditing alone did not capture task drift reliably in the pilot runs, as we describe next. The verifier agent is asked to run the oracle, challenge the proposed label, identify missing evidence, and check whether the proposed fix addresses the underlying defect. The full breakdown is in Appx. A.

Repair conventions. Once a defect is identified, the repair is not always mechanical, but involves human judgment (Zhu et al., 2025). We follow general conventions for specification and verification and task drift:

- **Instruction–verification mismatch**: any solution satisfying the task as written is accepted, and invalid solutions are rejected.
- **Task drift**: make the dependency reproducible where possible and remove any dependencies which do not let the oracle run reliably.

We use the following conventions for resource mismatches and task difficulty:

Environment Issues/Resource mismatches. We set a *feasibility floor*: a known-correct oracle solution and the verifier should run reliably within the benchmark budget. For resource-related failures, we adjust only the binding resource axis (wall-clock time, memory, or CPU count) to the factor closest to the original proposal or at least $1.5\times$ the observed requirement of the oracle solution and verifier.

This convention serves two purposes. First, it prevents false negatives caused by infrastructure noise: a correct solution should not fail because the verifier or oracle only barely fits within the budget, but we should leave enough slack for the problem to be solved. Second, setting the resource sets the task difficulty, and here we leave those as close to what the original task contributors intended.

An alternative would be to tune resource budgets per task. However, doing so would require repeated sweeps over any resource and would introduce an additional subjective design choice into the process. While the feasibility rule is conservative, it is an auditable convention.

Cross-cutting conventions. When tasks depend on live APIs, the difficulty can change after the audit. When a repair changes task difficulty, we document the change rather than tuning the task back to match its previous pass rate. Repairs are adjudicated in the open, and keyed to the defect category they address.

Workflow Evaluation We evaluate the audit workflow in the setting it was designed for: post-release benchmark maintenance. We used the workflow during Terminal-Bench 2.0 maintenance to surface candidate task defects, attach evidence, and propose repairs for maintainer review. The accepted repairs were then shipped as Terminal-Bench 2.1.

We evaluate the workflow along three dimensions. First, we ask whether the workflow is agent-agnostic: under the same rubric, do different auditors assign similar defect labels? Second, we ask whether this agreement persists when the workflow is instantiated with lower-cost auditor models. Third, we evaluate the workflow as a maintainer-in-the-loop triage system by comparing its flags against the final human-adjudicated TB2.1 repair documented in PR #53 (**?**).

As we described previously, the agents assign a task to a defect category. The defect categories were selected based on results from a 15-task pilot subset of Terminal-Bench 2.0 and approximately 1,500 TB2.0 execution trials.

To evaluate if the workflow is agent agnostic, we evaluated the workflow with two classes of agents: managed coding assistants and lower-cost auditor models. Claude Code/Opus 4.6 and Codex/GPT-5.4 reach agreement of $\kappa = 0.80$. The lower-cost Gemini 2.5 Flash and GPT-4o-mini auditors achieve $\kappa = 0.769$. These results suggest that the workflow is agent-agnostic and that different auditor pairs recover similar defect labels under the same rubric. An extended breakdown appears in Appx. A.

We next compare workflow outputs against the final Terminal-Bench 2.1 release repairs, treating the human-adjudicated repair set documented in PR #53 as retrospective ground truth (**?**). Against the final TB2.1 release, the workflow reaches 57.1% precision and 71.4% recall. Thus, the workflow is useful as a high-recall defect-discovery tool. When we break performance down by category, we find that disagreements among agents primarily stem from

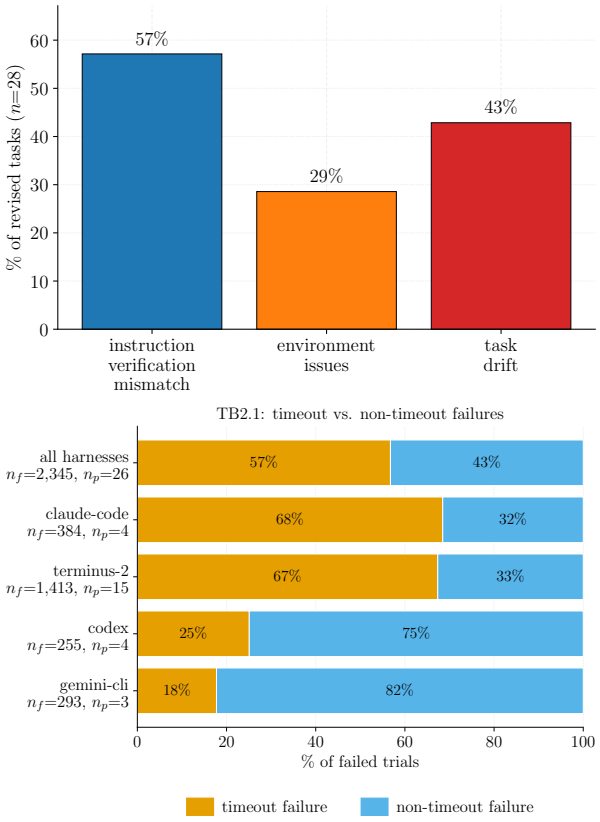


Figure 3. **Two complementary views of where task errors come from.** (a) Prevalence of defect categories in 28 tasks from Terminal Bench 2.0. (b) Prevalence of timeout failed trials in Terminal Bench for different agents (each harness includes different agent sets).

finer-grained attribution. For the local agents, at the category level, agreement is moderate, with instruction–verification mismatch ($\kappa = 0.60$) and environment issues ($\kappa = 0.59$), but the task drift is noisier ($\kappa = 0.21$–0.41). In turn besides rubrics, we expand the verifier to run complementary oracle calls that catch drift deterministically: every drift defect modifies the environment or solution code, so re-running the oracle solution surfaces the defect.

5. From Terminal Bench 2.0 to 2.1

Applying the audit workflow to Terminal-Bench 2.0 surfaced issues in 28 tasks, roughly a third of the benchmark. These consolidate defects found in prior audits, namely the 11 task repairs from Terminal-Bench Verified (Z.ai, 2026) and 5 further tasks addressed across various public issues and pull requests (Github, 2026a;b), together with 14 additional tasks that the audit newly identified as needing repair.

Fig. 3 shows the breakdown of the task defects in the 28 tasks that needed repair. The most common failure mode was instruction–verification mismatch (present in 60% of the failures). For example, on the task `query-optimize`, the instruction said not to modify the database, but the test checked a hash; an agent that touched and reverted the file failed even though the database was logically identical (Aleithan et al., 2024; Yu et al., 2025).

The second most common defect was task drift (40% of tasks). Tasks with internet access introduce external dependencies that change over time, as expected. In TB2.0 task contributors assigned resource budgets for each task, including number of CPUs and memory in CPU, based on human work. These estimates were too tight in several cases: the verifier itself did not have headroom to run after the agent's solution completed, producing failures that depended on the agent's resource use rather than its correctness. Furthermore, as other work has dicussed, different VM providers (Docker, Daytona) differ in how they isolate filesystems, enforce memory limits, and expose networks between agent and verifier. The audit raises each budget so the oracle runs with at least a $1.5\times$ slack on the binding axis, restoring a feasibility floor. The extended per-task taxonomy with secondary categories appears in Appx. A.

An audit of this size is dangerous: if agent performance reorders the leaderboard, every prior claim on TB2.0 is suspect, ours included. We test this directly in the next sections.

6. Results

The performance of an agent is a product of the model, harness and resources (Yang et al., 2024; Kapoor et al., 2025; Merrill et al., 2026). While we generally compare model and harness, here we show the impact of resources as well. We use TB2.1 to ask which conclusions are stable under changes to this measurement stack. First, we compare TB2.0 and TB2.1 at the default protocol to test whether repairs change accuracy and rankings. Second, we vary the time budget to test whether repairs change the efficiency frontier: which agents solve tasks, and how much budget they need to do so. Third, we ask how much of a fixed-budget harness gap is explained by speed under the timeout. Scalar leaderboards hide these distinctions because they collapse benchmark version, harness, and resource budget into one number.

6.1. The audit raises accuracy and changes close rankings

We evaluate 19 agents from multiple providers and compare the accuracy in both TB 2.0 and TB 2.1, as illustrated in Fig. 4). The correlation is high (Spearman $\rho = 0.958$, 95% CI $[0.881, 0.979]$), with an OLS fit of TB2.1 $= 2.6 + 1.03 \cdot$ TB2.0. The lift is uneven: Claude Code + Claude Opus 4.6 lifts $+7.3$ pp above the fit (rank $8 \rightarrow 5$), while Codex + GPT 5.4 sits 7.8 pp below the fit. Negative-residual agents concentrate below 50% on TB 2.0 accuracy, suggesting that weaker systems benefit less from the repairs. This aligns with the distribution of difficulty in the fixed tasks: 19 of the 28 repaired tasks were hard, 7 medium, and 2 easy.

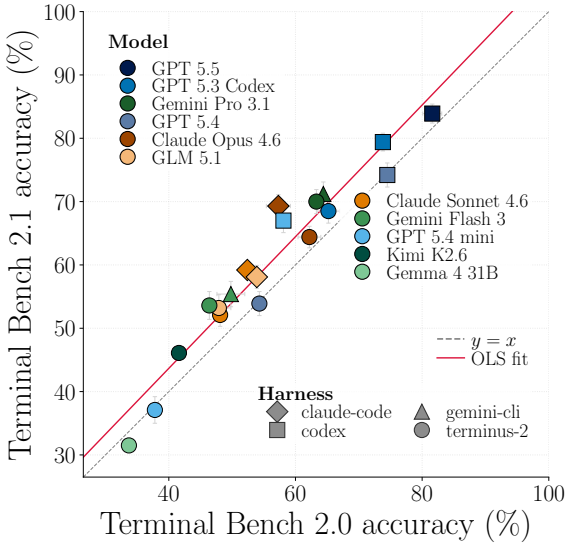


Figure 4. **Terminal Bench 2.1 changes accuracy unevenly across agents.** Spearman $\rho = 0.958$ with a task-bootstrap 95% CI of [0.881,0.979] across 19 agents.

Verifier fixes account for the largest score movements. In TB 2.1, five tasks increased from pass rates of at most 15.8% to at least 56.1% after verifier-only repairs, accounting for roughly 62.5% of the total absolute pass-rate change on the modified-task subset (full breakdown in Appx. C). These tasks fall under the instruction–verification mismatch: agents were often producing valid solutions, but TB 2.0 failed to recognize them. The remaining 23 modified tasks contribute smaller changes in both directions. In some cases, the audit made tasks stricter by removing false-positive credit or tightening the intended specification, lowering TB 2.1 pass rates relative to TB 2.0.

6.2. The audit shifts the efficiency frontier

Setting resources for executable agent benchmarks is non-trivial. A task score depends not only on whether an agent can solve the task, but also on whether it can solve it within the allocated resources. Fig. 3 shows that most weak agent–model failures are due to timeout rather than capability.

We therefore evaluate the test-time scaling properties of these models. For each agent–model pair, we increase the wall-clock budget up to 32× the original task timeout and measure how many tasks are solved as the budget increases. This larger budget is an upper bound: agents can exit once they finish, so they do not need to consume the full allocation. At this budget, timeout errors disappear for the agent–model pairs considered here, letting us ask which failures remain once we remove time as a binding constraint.

Fig. 5 shows the coverage for six agent model pairs using Terminus-2; GLM 5.1 with Claude-Code is included as a reference point because it lies on the efficient frontier. The solid curves show TB2.1 and the dashed curves show TB2.0. Each curve should be read in two ways. Its left–right position shows efficiency: a curve that rises earlier corresponds to an agent that solves tasks with less time or fewer output tokens. Its final height shows coverage at the largest budget: if the curve flattens below 1.0, then giving more budget does not recover the remaining tasks.

First, agents differ substantially in efficiency. GLM 5.1 reaches high coverage at much smaller budgets than the other agents, so its curve sits farther to the left. GPT 5.4 reaches higher final coverage, but requires more budget to get there. The smaller open models, including Qwen 3.5 9B, Qwen 3.5 27B, and Gemma 4 31B, solve many tasks as the budget increases, but their curves flatten at lower coverage. Since the wall-clock sweep removes timeout errors in these runs, the remaining failures are not timeout artifacts, but tasks the models cannot solve.

Second, the audit changes these curves unevenly. For GLM 5.1, TB2.1 credits substantially more solutions than TB2.0, shifting the curve upward. For the Qwen pairs, by contrast, the TB2.0 and TB2.1 curves are nearly unchanged. Thus, benchmark repair does not simply add a constant score increase. It changes which solutions are credited, and therefore can change which agents appear efficient at a given budget.

Harness gaps are speed gaps. Prior work has shown that harnesses and agent-computer interfaces affect agent performance (Yang et al., 2024; Kapoor et al., 2025; Merrill et al., 2026). We ask how much of a fixed-budget harness gap disappears when the slower harness is moved along the efficiency frontier?

Fig. 6 uses within-vendor harness swaps as a diagnostic. At the default time, codex gives much higher scores than terminus-2 for OpenAI models. Harness choice moves GPT 5.4's accuracy by 20 pp, rivaling between-model gaps. However, most of the harness gap is recovered by raising the time, so the harness gap is a budget artifact rather than a capability difference.

The results here show that a fixed resource budget is an arbitrary operating point, and these results show the choice is not innocuous: it can change efficiency conclusions for individual models. This supports setting a principled floor and testing sensitivity to the budget, rather than calibrating to any one choice.

7. Discussion

Continuous validation. Our audit pipeline surfaced 28 task-side defects in TB2.0, including 12 cases of post-release drift in external dependencies. Drift is by definition a

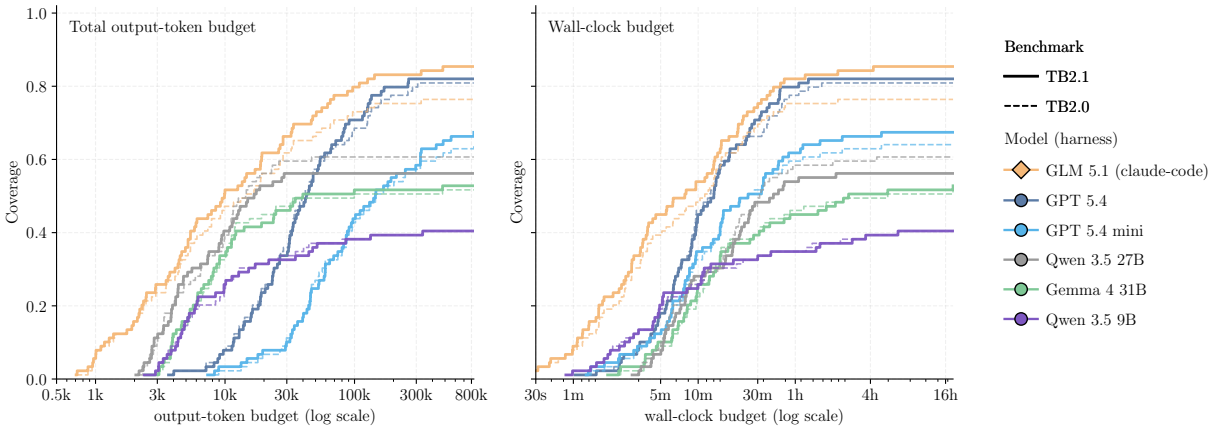


Figure 5. **Efficiency differences across different agents.** Coverage changes as resources are unlocked until they are not a bottleneck.

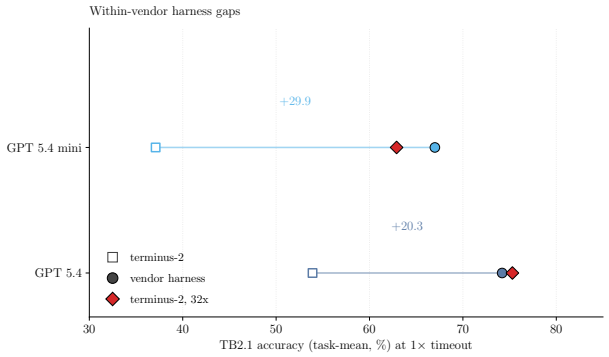


Figure 6. **Within-vendor harness gaps are a timeout effect on the OpenAI cohort.** Per-agent performance between preferred and base harness at 1× timeout; red diamonds mark available `terminus-2` results at 32× timeout. The plotted OpenAI points recover substantially under larger wall-clock budgets, so the measured gap is timeout-sensitive rather than a pure model-capability comparison. See Sec. 6.2.

recurring failure mode: more dependencies will rot between this paper's submission and its publication. This is the case for version control rather than one-shot validation. Efforts like SWE-Bench Verified and WebArena Verified address defects at release time but imply a terminal validated state that does not exist for benchmarks tied to live infrastructure. Numbered, dated revisions with a `CHANGELOG.md` fit this setting better. LiveCodeBench (Jain et al., 2025) already operates this way for contamination control; the same discipline applies to task-side defects.

Performance preservation under audit. Across 19 agents, the maximum difference in rank ordering is 3. Agent-level score shifts have median 5.3 pp (IQR 2.3–6.8). The largest gains concentrate on instruction–verification mismatches, which affected the largest most the most.

The audit also shows the sensitivity of efficiency conclusions to different design choices, and how setting a feasibility floor is an affordable fix.

Overall, this work provides (i) an audit methodology that surfaces and repairs benchmark defects (ii) the TB2.1 artifact, demonstrating that audit-driven repairs can preserve agent rank ordering while shifting absolute scores; and (iii) evidence that, on TB2.1, both task definitions and resource budgets shape published scores in ways current reporting practice does not control for. Finally, because this pipeline can run continuously, Terminal Bench 2.1 is a checkpoint rather than a terminal state.

References

Aleithan, R., Xie, H., Lo, D., and Wang, S. SWE-Bench+: Enhanced coding benchmark for LLMs. *arXiv preprint*, 2024. URL https://arxiv.org/abs/2410.06992.

Anthropic. Claude code: An agentic coding tool. Software documentation, 2025. URL https://docs.anthropic.com/en/docs/agents-and-tools/claude-code/overview. CLI reference: https://docs.anthropic.com/en/docs/agents-and-tools/claude-code/cli-reference.

Anthropic. Introducing Claude Opus 4.6. Anthropic announcement and system card, February 5 2026a. URL https://www.anthropic.com/news/claude-opus-4-6. Terminal-Bench 2.0 reported on Terminus-2 harness (Codex CLI for OpenAI models), 1× guaranteed / 3× ceiling resource allocation, 5–15 samples per task across staggered batches.

Anthropic. Introducing Claude Sonnet 4.6. Anthropic announcement and system card, February 17 2026b. URL https://www.anthropic.com/news/claude-sonnet-4-6.

Chen, M., Tworek, J., Jun, H., Yuan, Q., et al. Evaluating large language models trained on code. *arXiv preprint*, 2021. URL https://arxiv.org/abs/2107.03374.

Cursor Research Team. Composer 2: A fast frontier coding model built for the agent loop. Cursor technical report, 2026. URL https://cursor.com/resources/Composer2.pdf. Reports Terminal-Bench 2.0 with mixed harnesses (Claude Code for Anthropic models, Simple Codex for OpenAI, Harbor for Composer-2), averaged over 5 iterations.

Github, T. Terminal-bench: GitHub issue and pull-request tracker, 2026a. URL https://github.com/harbor-framework/terminal-bench. Accessed May 2026; community-reported defects against the public TB and TB2.0 task sets.

Github, T. Terminal-bench 2.0: GitHub issue and pull-request tracker, 2026b. URL https://github.com/harbor-framework/terminal-bench-2. Accessed May 2026; community-reported defects against the public TB2.0 task set.

Google. Gemini CLI: An open-source AI agent for the terminal. Open-source software (Apache 2.0), 2025. URL https://github.com/google-gemini/gemini-cli. Documentation: https://geminicli.com/docs/; tool reference: https://geminicli.com/docs/tools/.

Google DeepMind. Gemini: A family of highly capable multimodal models, 2024. URL https://arxiv.org/abs/2312.11805.

Google DeepMind. Gemma 4: Open multimodal models. Model card and announcement; https://ai.google.dev/gemma/docs/core/model_card_4, 2026. URL https://deepmind.google/models/gemma/gemma-4/. Released April 2026. Sizes: E2B, E4B, 26B MoE, 31B Dense; 256K context. We use the 31B Dense variant in the broader 20-pair audit pool.

Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., et al. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2403.07974.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.06770.

Kapoor, S., Stroebl, B., Kirgis, P., Nadgir, N., Siegel, Z. S., Wei, B., Xue, T., Chen, Z., et al. Holistic agent leaderboard: The missing infrastructure for AI agent evaluation. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2510.11977.

LangChain. Langsmith: Ai agent and llm observability platform. https://www.langchain.com/langsmith/observability, 2026. Accessed: 2026-06-30.

Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., et al. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2308.03688.

Meng, K., Huang, V., Steinhardt, J., and Schwettmann, S. Introducing docent, 2025. URL https://transluce.org/introducing-docent. Transluce AI Technical Report.

Merrill, M. A., Shaw, A. G., et al. Terminus: The terminal-bench baseline agent, 2025. URL https://github.com/terminal-bench/terminus. Baseline agent scaffold for Terminal-Bench.

Merrill, M. A., Shaw, A. G., Carlini, N., Li, B., Raj, H., Bercovich, I., et al. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL https://arxiv.org/abs/2601.11868.

Moonshot AI. Kimi K2: Open agentic intelligence. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2507.20534. 1.04T-parameter MoE; 32B active. Open-weight agentic coding model. We use Kimi K2.6 in the broader 20-pair audit pool.

Moonshot AI. Kimi K2 Thinking model card. Hugging Face, 2026. URL https://huggingface.co/moonshotai/Kimi-K2-Thinking. Terminal-Bench reported with the default Terminus-2 framework and a custom JSON parser; other coding tasks reported on an in-house harness derived from SWE-agent. All scores averaged over 5 independent runs.

OpenAI. GPT-4 technical report, 2024a. URL https://arxiv.org/abs/2303.08774.

OpenAI. Introducing SWE-bench Verified. OpenAI blog post, 2024b. URL https://openai.com/index/introducing-swe-bench-verified/. Human-validated 500-task subset of Jimenez et al. (2024).

OpenAI. Codex CLI: A lightweight coding agent that runs in the terminal. Software platform, 2025a. URL https://openai.com/codex. CLI reference: https://developers.openai.com/codex/cli/reference/.

OpenAI. Ghosts in the codex machine. Technical report, 2025b. OpenAI technical report.

OpenAI. Harness engineering: leveraging Codex in an agent-first world. https://openai.com/index/harness-engineering/, February 2026. Accessed: 2026-05-05.

Qwen Team. Qwen technical report, 2024. URL https://arxiv.org/abs/2309.16609.

Qwen Team. Qwen3.6 model card. Hugging Face, 2026. URL https://huggingface.co/Qwen/Qwen3.6-27B. Terminal-Bench 2.0 reported on Harbor/Terminus-2; 3h timeout, 32 CPU/48 GB RAM, max_tokens=80K, 256K context, averaged over 5 runs. Different harnesses for SWE-Bench (internal scaffold), SkillsBench (OpenCode), and NL2Repo (Claude Code).

Recht, B., Roelofs, R., Schmidt, L., and Shankar, V. Do ImageNet classifiers generalize to ImageNet? *Proceedings of ICML*, 2019. URL https://arxiv.org/abs/1902.10811.

Segato, G. Quantifying infrastructure noise in agentic coding evals. Anthropic Engineering Blog, 2026. URL https://www.anthropic.com/engineering/infrastructure-noise.

ServiceNow Research. WebArena Verified: Reliable evaluation for web agents. Software release, 2026. URL https://github.com/ServiceNow/webarena-verified. Audited 812-task subset of Zhou et al. (2024).

SWE-agent Team. Mini swe agent. Code repository, 2025. URL https://github.com/SWE-agent/mini-swe-agent.

Wang, X., Chen, B., Adelt, Z., Shi, Y., Liu, B., Yang, S., and Neubig, G. Openhands: An open platform for AI software developers as generalist agents, 2025. URL https://arxiv.org/abs/2407.16741.

White, C., Dooley, S., Roberts, M., Pal, A., Feuer, B., Jain, S., Shwartz-Ziv, R., Jain, N., Saifullah, K., Dey, S., Shubh-Agrawal, Sandha, S. S., Naidu, S. V., Hegde, C., LeCun, Y., Goldstein, T., Neiswanger, W., and Goldblum, M. Livebench: A challenging, contamination-free LLM benchmark. In *The Thirteenth International Conference on Learning Representations*, 2025.

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O. Swe-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://arxiv.org/abs/2405.15793.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2210.03629.

Yu, B. et al. UTBoost: Rigorous evaluation of coding agents on SWE-Bench. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. URL https://arxiv.org/abs/2506.09289.

Z.ai. Terminal-bench 2.0 verified: Instruction & environment fix version. Hugging Face dataset, https://huggingface.co/datasets/zai-org/terminal-bench-2-verified, 2026. Documents 89 environment fixes and 11 instruction fixes; reports a 5.5 pp shift on Claude Opus 4.5 from environment fixes alone. Companion to GLM-5 (Zeng et al., 2026).

Zeng, A., Lv, X., Hou, Z., Du, Z., Zheng, Q., Chen, B., Yin, D., et al. GLM-5: from vibe coding to agentic engineering. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2602.15763.

Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., et al. WebArena: A realistic web environment for

building autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2307.13854.

Zhu, Y., Jin, T., Pruksachatkun, Y., Zhang, A., Liu, S., Cui, S., et al. Establishing best practices for building rigorous agentic benchmarks. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2507.02825.

Appendix Contents

A Audit workflow Details **12**

A.1 Continuous-validation pipeline 12

A.2 Audit Workflow Evaluation 12

B Terminal-Bench Benchmark Evaluation **12**

B.1 Models evaluated 12

B.2 Harness Evaluation 12

B.3 Agent configuration overrides 13

C Supplementary results **15**

C.1 Per-task pass-rate changes (TB2.0 $\rightarrow$ TB2.1) 15

C.2 Agent Failure decomposition 15

A. Audit workflow Details

A.1. Continuous-validation pipeline

As benchmark users continue to open issues and pull requests, maintainers need a lightweight way to check whether proposed changes preserve task validity. We implement this audit workflow with GitHub Actions that can be invoked manually or triggered automatically on pull requests for a given task.

The pipeline exposes two checks which leverage:

- `/audit` call the auditor agent to go over the task instructions and available trial logs.
- `/oracle` builds the task environment and runs the reference solution, checking that a known-correct solution still passes.

Propose-only. Although agents could be given permission to propose fixes and merge pull requests automatically, we deliberately restrict the workflow to propose-only operation. Each run is capped at five repair iterations, and the workflow grants pull-requests but prevents agents from directly changing repository state. Instead, proposed fixes are emitted as maintainer-reviewable diffs in `audit/changes/<task>.diff`.

The audit rubrics are integrated in https://github.com/harbor-framework/benchmark-template.

A.2. Audit Workflow Evaluation

The workflow is agent-agnostic: the evidence and auditor agents can be instantiated either with managed, agent-first coding assistants, such as Claude Code or Codex, or with cheaper models suitable for local evaluation. We evaluate this flexibility by measuring agreement among auditor agents on the binary audit decision, i.e., whether each task is broken or clean. We find that the managed coding assistants Claude Code (powered by Opus 4.6) and Codex (powered by GPT-5.4) exhibit high agreement of ($\kappa = 0.80$). Agreement remains high even for local, low-cost auditor agents which achieve $\kappa = 0.769$ as shown in Table 1.

These results suggest that the binary audit decision is stable across capable auditor implementations, while weaker agents may require additional calibration or human review.

Table 1. **Inter-judge agreement.** Cohen's κ between pairs of judges on the binary flag (flagged vs. audit-clean) over all 89 TB2.0 tasks.

Agent A	Agent B	Agree	Disagree	κ
gpt-4o-mini	gpt-4o	69	20	0.550
gpt-4o-mini	gemini-2.5-flash	79	10	0.769
gpt-4o	gemini-2.5-flash	69	20	0.549

When we break performance down by issue category, we find that disagreements among agents primarily stem from finer-grained attribution rather than from the binary triage decision. On the binary classifier, flagged versus clean, cross-provider agreement reaches Cohen's $\kappa = 0.77$ even for weak agents. At the category level, agreement remained moderate, with instruction–verification mismatch ($\kappa = 0.60$) and environment issues ($\kappa = 0.59$), but task drift is noisier ($\kappa = 0.21$–0.41). However, for resource-related failures, checking that the `/oracle` solution runs fully captures this type of defect.

Per-task map to error categories Table 2 shows the primary failure model assigned to each task found to be broken. Table 3 shows the final 28 revised tasks, and which component in the task was updated for TB 2.1.

B. Terminal-Bench Benchmark Evaluation

B.1. Models evaluated

The evidence data included trials from the leaderboard coming from 13 agents as outlined in Table 4.

B.2. Harness Evaluation

Each model was evaluated with two harnesses: a common baseline harness, `terminus-2` (Merrill et al., 2026), and the model provider's proprietary agent harness. The latter includes Anthropic's Claude Code CLI (Anthropic, 2025), OpenAI's Codex CLI (OpenAI, 2025a; 2026), and Google's Gemini CLI (Google, 2025).

Table 2. **Failure-modes among the 28 tasks revised in TB2.1.**

Instruction–verification mismatch (16)	Resource mismatch (8)	Task drift (12)
adaptive-rejection-sampler, build-pmars, caffe-cifar-10, configure-git-webserver, filter-js-from-html, fix-git, hf-model-inference, install-windows-3.11, mteb-leaderboard, mteb-retrieve, polyglot-c-py, polyglot-rust-c, query-optimize, sam-cell-seg, torch-tensor-parallelism, train-fasttext	caffe-cifar-10, compile-compcert, crack-7z-hash, filter-js-from-html, gpt2-codegolf, query-optimize, torch-pipeline-parallelism, torch-tensor-parallelism	adaptive-rejection-sampler, build-pmars, build-pov-ray, extract-moves-from-video, financial-document-processor, make-doom-for-mips, mcmc-sampling-stan, mteb-leaderboard, mteb-retrieve, overfull-hbox, protein-assembly, rstan-to-pystan

Table 3. **Breakdown of the 28 TB 2.0 tasks revised in TB 2.1, by the stage of the task definition that required repair.**

Task	Instr.	Env.	Oracle	Tests	Time	Mem.	CPUs
adaptive-rejection-sampler	✓		✓				
build-pmars	✓			✓			
build-pov-ray			✓				
caffe-cifar-10	✓	✓		✓	✓	✓	✓
crack-7z-hash					✓	✓	
compile-compcert		✓					✓
configure-git-webserver			✓	✓			
extract-moves-from-video	✓	✓					
filter-js-from-html	✓				✓	✓	
financial-document-processor			✓				
fix-git		✓		✓			
gpt2-codegolf						✓	
hf-model-inference				✓			
install-windows-3.11	✓			✓			
make-doom-for-mips			✓				
mcmc-sampling-stan				✓			
mteb-leaderboard	✓	✓				✓	
mteb-retrieve	✓	✓					
overfull-hbox			✓				
polyglot-c-py				✓			
polyglot-rust-c				✓			
protein-assembly		✓	✓				
query-optimize	✓				✓		
rstan-to-pystan				✓			
sam-cell-seg	✓						
torch-pipeline-parallelism						✓	
torch-tensor-parallelism	✓					✓	
train-fasttext				✓			

B.3. Agent configuration overrides

Table 6 lists the agent configurations. Rows marked "default" use the the harness-level defaults documented in Table 5.

We note that `terminus-2` triggers proactive summarization when the free context drops below a threshold (default 8K tokens). We leave the default for frontier-API models and GLM 5.1, where a single turn rarely exceeds 8K tokens. For the open-weight long-context models (Gemma 4 31B and Kimi K2.6, both with 254K context windows), we raise the threshold to 131K tokens after observing trials where summarization fired too late to leave room for one more turn.

The Gemma 4 31B and Kimi K2.6 rows in Table 6 are collapsed for readability; the full sampling settings are:

- **Gemma 4 31B:** temperature 1.0, interleaved thinking on, max input tokens 253,952, max output tokens 8,192, proactive-summarization threshold 131,072, `top_p`=0.95, `top_k`=20, `presence_penalty`=1.5.
- **Kimi K2.6:** temperature 0.6, interleaved thinking on, max input tokens 253,952, max output tokens 8,192, proactive-summarization threshold 131,072, `top_p`=0.95, `top_k`=20.

Table 4. **Models evaluated.** One row per model entry used in the agent-rank comparison or compute-scaling analyses. CTX values in italics are pinned smaller by the harness config; plain values are the vendor maximum.

Model	Provider	Route	Ctx
Claude Opus 4.6 (Anthropic, 2026a)	Anthropic	Anthropic API	200K
Claude Sonnet 4.6 (Anthropic, 2026b)	Anthropic	Anthropic API	200K
GPT 5.3 Codex (OpenAI, 2025b)	OpenAI	OpenAI API	400K
GPT 5.4 (OpenAI, 2025b)	OpenAI	OpenAI API	400K
GPT 5.4 mini (OpenAI, 2025b)	OpenAI	OpenAI API	400K
GPT 5.5 (OpenAI, 2025b)	OpenAI	OpenAI API	400K
Gemini Flash 3 (Google DeepMind, 2024)	Google	Google API	1M
Gemini Pro 3.1 (Google DeepMind, 2024)	Google	Google API	1M
Gemma 4 31B (Google DeepMind, 2026)	Google	Together.ai	256K *(131K)*
GLM 5.1 (Zeng et al., 2026)	Z.AI	Anthropic-shape	200K
Kimi K2.6 (Moonshot AI, 2025; 2026)	Moonshot AI	Together.ai	256K *(131K)*
Qwen 3.5 9B (Qwen Team, 2024; 2026)	Alibaba	Together.ai	254K *(131K)*
Qwen 3.5 27B (Qwen Team, 2024; 2026)	Alibaba	Together.ai	254K

Table 5. **Harnesses evaluated.**

Harness	Scaffold	Source / defaults
`terminus-2`	single-agent ReAct (Yao et al., 2023)	Terminal-Bench 2.0 baseline (Merrill et al., 2025; Github, 2026a); including JSON-action parser, shell + file I/O tools. **Defaults:** `enable_summarize=True`, `proactive_summarization_threshold=8000`, `temperature=1.0` for thinking-capable models.
`claude-code`	provider-managed agentic CLI	Anthropic Claude Code, headless (Anthropic, 2025); self-managed tool palette + context compaction. **Defaults in our runs:** `reasoning_effort=high`, adaptive thinking on.
`codex`	provider-managed agentic CLI	OpenAI Codex CLI, non-interactive (OpenAI, 2025a; 2026); native Responses-API tool calls, managed context. `use_responses_api=True`.
`gemini-cli`	provider-managed agentic CLI	Google Gemini CLI, Apache 2.0 (Google, 2025); shell + file I/O tools, managed context.

Table 6. **Agent Model Configuration.** "Default" means the harness-level defaults of Table 5 apply unchanged. Extra defaults listed below.

Agent	Model	Overrides vs. harness default
claude-code	Claude Opus 4.6	`reasoning_effort=high`
claude-code	Claude Sonnet 4.6	`reasoning_effort=high`
claude-code	GLM 5.1	default
codex	GPT 5.3 Codex	`reasoning_effort=xhigh`
codex	GPT 5.4	`reasoning_effort=xhigh`
codex	GPT 5.4 mini	`reasoning_effort=xhigh`
codex	GPT 5.5	`reasoning_effort=xhigh`, `temperature=1.0`
gemini-cli	Gemini Flash 3	default
gemini-cli	Gemini Pro 3.1	default
terminus-2	Claude Opus 4.6	`reasoning_effort=high`, `T=1`
terminus-2	Claude Sonnet 4.6	`reasoning_effort=high`, `T=1`
terminus-2	GLM 5.1	`T=1.0`, `thinking.type=enabled`
terminus-2	GPT 5.3 Codex	`reasoning_effort=xhigh`, `use_responses_api=True`
terminus-2	GPT 5.4	`reasoning_effort=xhigh`, `use_responses_api=True`
terminus-2	GPT 5.4 mini	`reasoning_effort=xhigh`, `use_responses_api=True`
terminus-2	GPT 5.5	`reasoning_effort=xhigh`, `T=1`, `use_responses_api=True`
terminus-2	Gemini Flash 3	`reasoning_effort=high`
terminus-2	Gemini Pro 3.1	`reasoning_effort=high`
terminus-2	Gemma 4 31B	`T=1.0`, `interleaved_thinking=True`; open-weight pin block (see text)
terminus-2	Kimi K2.6	`T=0.6`, `interleaved_thinking=True`; open-weight pin block (see text)

C. Supplementary results

C.1. Per-task pass-rate changes (TB2.0 → TB2.1)

Table 7 reports the per-task pass-rate change from TB2.0 to TB2.1, averaged across all agents. After the changes no task remains unsolved. The largest regression is in task `overfull-hbox` due to an insufficient verifier.

Table 7. **Changed-task pass rates.** Rates aggregated across all agent model pairs evaluated.

Task	TB2.0	TB2.1
adaptive-rejection-sampler	31.7%	58.3%
build-pmars	75.0%	81.7%
build-pov-ray	75.0%	68.3%
caffe-cifar-10	5.0%	69.2%
compile-compcert	45.0%	48.3%
configure-git-webserver	46.7%	50.0%
crack-7z-hash	75.0%	86.7%
extract-moves-from-video	8.3%	13.3%
filter-js-from-html	0.0%	6.7%
financial-document-processor	61.7%	56.7%
fix-git	98.3%	95.0%
gpt2-codegolf	15.0%	18.3%
hf-model-inference	88.3%	86.7%
install-windows-3.11	0.0%	23.3%
make-doom-for-mips	6.7%	1.7%
mcmc-sampling-stan	75.0%	71.7%
mteb-leaderboard	28.3%	36.7%
mteb-retrieve	15.0%	41.7%
overfull-hbox	53.3%	36.7%
polyglot-c-py	5.0%	80.0%
polyglot-rust-c	5.0%	66.7%
protein-assembly	20.0%	15.0%
query-optimize	50.0%	43.3%
rstan-to-pystan	58.3%	58.3%
sam-cell-seg	0.0%	53.3%
torch-pipeline-parallelism	8.3%	10.0%
torch-tensor-parallelism	15.0%	66.7%
train-fasttext	5.0%	3.3%

C.2. Agent Failure decomposition

Agentic evaluation infrastructure is still a work in progress: runner failures, provider interruptions, authentication errors, sandbox crashes, and setup timeouts can all occur before the agent has produced a task-level outcome. To avoid mixing these infrastructure failures with model behavior, this analysis uses a conservative whitelist-filtering approach. The table builder retains only trials whose terminal state can be interpreted as an agent outcome: a pass, a completed wrong answer, or an `AgentTimeoutError`. Other upstream infrastucture errors are filtered before the decomposition is computed. However, we note that these filtered infrastructure trials account for roughly $27.\%$ of upstream trials.

Fig. 7 decomposes each whitelisted trial into one of three mutually exclusive outcomes: a pass (`reward`$=1$), a completed but incorrect answer (`reward`$=0$), or an agent timeout (`AgentTimeoutError`). At fixed budget, the weakest agents are dominated by timeouts rather than by scored wrong answers. For example, the Qwen 3.5 9B, Gemma 4 31B, Kimi K2.6, and weakest GPT 5.4 mini. Thus, for these cells, the benchmark often censors the run before a final answer can be scored. These results motivate the budget sweeps in Sec. 6.2.

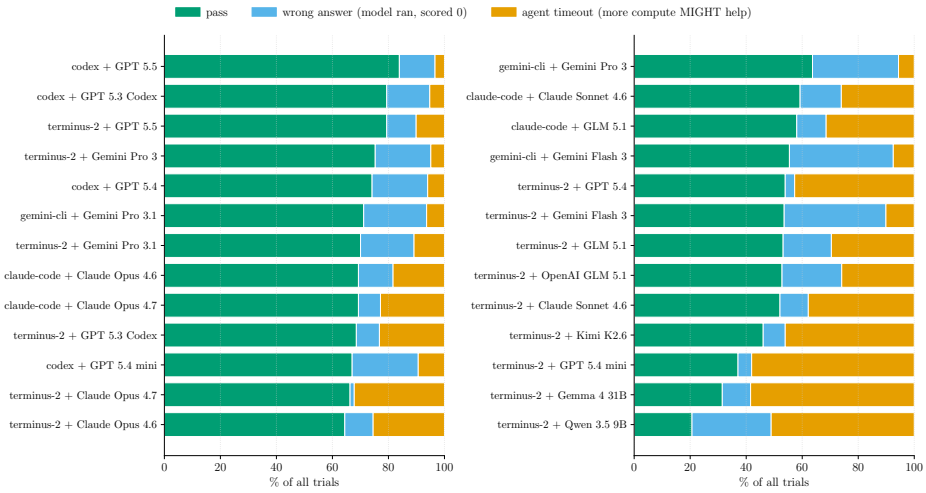


Figure 7. **Trial outcomes by agent.** Each horizontal bar shows, for one agent, the fraction of trials that passed (green), returned a clean wrong answer (sky blue), or hit `AgentTimeoutError` (amber).